

NP, NP-Hard and NP-Complete Problems

1 Introduction

Why do we study NP Problems?

In algorithm analysis, we want to know whether a problem can be solved efficiently. A problem is considered **efficiently solvable** if it has an algorithm whose running time is polynomial, such as:

$$O(n), \quad O(n^2), \quad O(n^3), \quad O(n^k)$$

But many important problems do not have known polynomial-time algorithms. These problems lead us to the study of:

P, NP, NP-hard, NP-complete.

Main Idea

P contains problems that can be solved quickly.

NP contains problems whose given solutions can be verified quickly.

NP-complete problems are the hardest problems inside NP.

2 Optimization Problem and Decision Problem

Optimization Problem

An **optimization problem** asks for the best possible solution among all valid solutions. Usually, it asks for:

- the **maximum**,
- the **minimum**,
- the **largest**,
- the **smallest**.

Example of Optimization Problem

In the **Vertex Cover optimization problem**, the goal is:

Find a minimum vertex cover.

In the **Independent Set optimization problem**, the goal is:

Find a maximum independent set.

Decision Problem

A **decision problem** asks a question whose answer is only:

Yes or **No**

In complexity theory, NP-completeness is defined for **decision problems**.

Optimization to Decision Version

Minimum vertex cover	→	Is there a vertex cover of size at most k ?
Maximum independent set	→	Is there an independent set of size at least k ?
Largest clique	→	Is there a clique of size at least k ?

Important Keywords

- **Minimum-type problem** usually becomes: **at most** k .
- **Maximum-type problem** usually becomes: **at least** k .

Examples:

Vertex Cover → at most k
Independent Set → at least k

3 Class P

Definition of P

The class **P** is the set of all decision problems that can be solved in polynomial time by a deterministic algorithm.

That means if a problem belongs to **P**, then there exists an algorithm that solves it in:

$$O(n^k)$$

for some constant k .

Easy Meaning

A problem is in **P** if we can find the answer efficiently.

Examples of Problems in P

- Searching in an array
- Sorting
- Shortest path problem
- Minimum spanning tree
- Checking whether a number is even or odd

4 Class NP

Definition of NP

The class **NP** is the set of all decision problems whose solutions can be verified in polynomial time.

Easy Meaning

A problem is in **NP** if:

- someone gives us a possible solution,
- we can check whether that solution is correct in polynomial time.

Certificate and Verifier

For a decision problem L , we say $L \in \mathbf{NP}$ if there exists:

- a **certificate** or proof y ,
- a polynomial-time **verifier** V ,

such that:

$$x \in L \iff V(x, y) = \mathbf{Yes}$$

Example

Problem:

Does graph G have a vertex cover of size at most k ?

Certificate:

C = a set of vertices

Verification:

- Check whether $|C| \leq k$.
- Check whether every edge has at least one endpoint in C .

This checking can be done in polynomial time. So, Vertex Cover is in **NP**.

Common Mistake

NP does not mean **Non-Polynomial**.

It means:

Nondeterministic Polynomial Time

In simple words, NP problems may be hard to solve, but their given solutions are easy to verify.

5 Relationship Between P and NP

Basic Relationship

Every problem that can be solved quickly can also be verified quickly.

Therefore:

$$\mathbf{P} \subseteq \mathbf{NP}$$

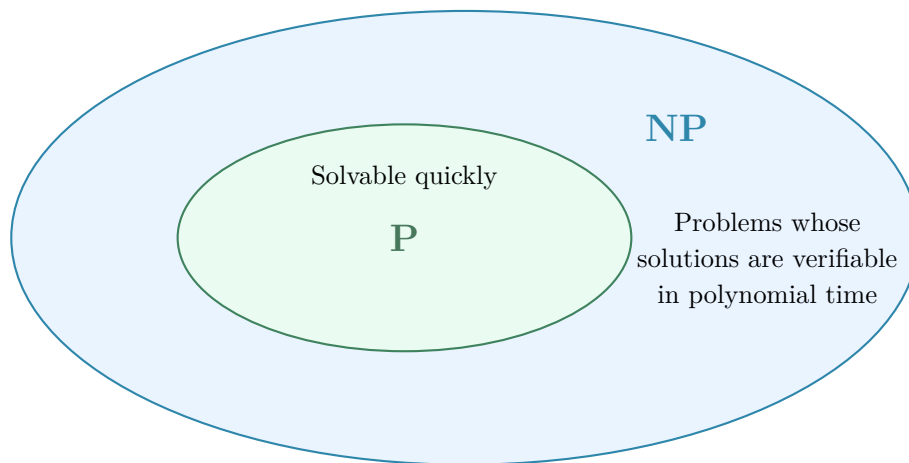


Figure: P is contained inside NP .

The Famous P vs NP Question

The most famous open question in computer science is:

$$P \stackrel{?}{=} NP$$

Meaning:

Can every quickly verifiable problem also be solved quickly?

6 Polynomial-Time Reduction

Definition

A problem A is polynomial-time reducible to problem B , written as:

$$A \leq_p B$$

if any instance of A can be transformed into an instance of B in polynomial time.

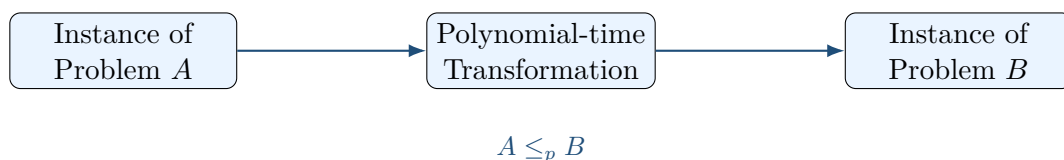
Meaning of Reduction

If:

$$A \leq_p B$$

then problem B is at least as hard as problem A .

Because solving B would also allow us to solve A .



Correct Direction of Reduction

To prove a new problem X is NP-hard, reduce a known NP-complete problem Y to X :

$$Y \leq_p X$$

Do **not** reduce X to Y .

7 NP-Hard Problems

Definition of NP-Hard

A problem H is called **NP-hard** if every problem in **NP** can be reduced to H in polynomial time.

Formally:

$$\forall L \in \mathbf{NP}, \quad L \leq_p H$$

Easy Meaning

An NP-hard problem is at least as hard as every problem in NP.

Important Point

An NP-hard problem does not need to be in NP.

So an NP-hard problem may be:

- a decision problem,
- an optimization problem,
- even a problem whose solution cannot be verified in polynomial time.

8 NP-Complete Problems

Definition of NP-Complete

A problem C is called **NP-complete** if it satisfies both conditions:

1. $C \in \mathbf{NP}$
2. C is NP-hard

NP-Complete Condition

$$C \in \mathbf{NP-complete} \iff (C \in \mathbf{NP}) \text{ and } (\forall L \in \mathbf{NP}, L \leq_p C)$$

Easy Meaning

NP-complete problems are the hardest problems inside NP.

If any NP-complete problem can be solved in polynomial time, then:

$$\mathbf{P} = \mathbf{NP}$$

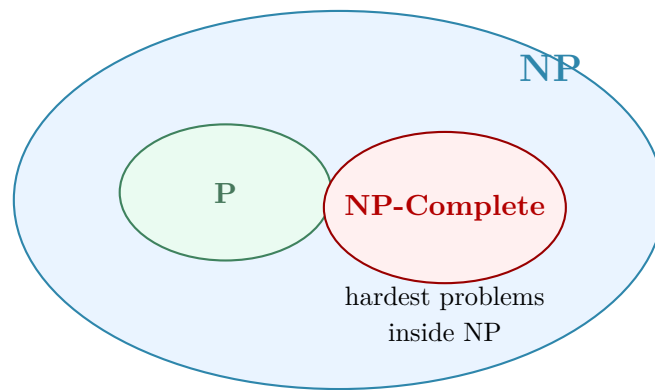


Figure: NP-complete problems are inside NP.

9 How to Prove a Problem is NP-Complete

Standard Proof Method

To prove a problem X is NP-complete:

1. Prove that $X \in \mathbf{NP}$.
2. Choose a known NP-complete problem Y .
3. Show a polynomial-time reduction:

$$Y \leq_p X$$
4. Therefore, X is NP-hard.
5. Since $X \in \mathbf{NP}$ and X is NP-hard, X is NP-complete.

Exam Warning

The direction of reduction is very important.

Correct:

Known NP-complete problem $\leq_p$ New problem

Wrong:

New problem $\leq_p$ Known NP-complete problem

10 Comparison Table

Class	Meaning	Key Idea
P	Problems solvable in polynomial time	Easy to solve
NP	Problems verifiable in polynomial time	Easy to check given answer
NP-hard	At least as hard as every NP problem	May not be inside NP
NP-complete	Both NP and NP-hard	Hardest problems inside NP

11 Exam Tips

How to Identify the Problem Type

- If the answer is **Yes** or **No**, it is a **decision problem**.
- If it asks for minimum or maximum, it is an **optimization problem**.
- If a solution can be checked quickly, the problem is in **NP**.
- If every NP problem can reduce to it, it is **NP-hard**.
- If it is both in NP and NP-hard, it is **NP-complete**.

12 Final Summary

One-box Revision Summary

P = problems solvable in polynomial time

NP = problems verifiable in polynomial time

P $\subseteq$ **NP**

NP-hard = at least as hard as every problem in NP

NP-complete = NP-hard and also in NP

$A \leq_p B \Rightarrow B$ is at least as hard as A

To prove X NP-complete: prove $X \in NP$ and reduce known NPC problem to X

Minimum problems usually use at most k

Maximum problems usually use at least k